# Mathematical Foundations for Guidance and Navigation

## Part 2. Matrix Fundamentals, Differential Calculation and Jacobians for Navigation

Dr. Weisong Wen, Assistant Professor
Trustworthy AI and Autonomous Systems Laboratory (TAS Lab), The Hong Kong Polytechnic University September 11, 2025

# Course Overview: Matrix Fundamentals for

- **Why study matrix mathematics?**
  - Navigation algorithms rely heavily on matrix operations
  - Sensor data fusion requires matrix manipulations
  - Advanced estimation methods use matrix decompositions
  - Matrix properties determine system stability and accuracy

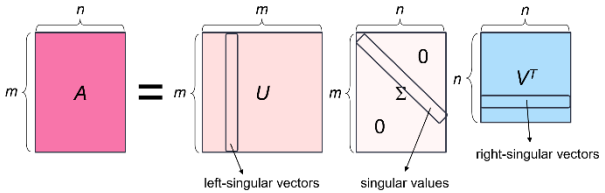


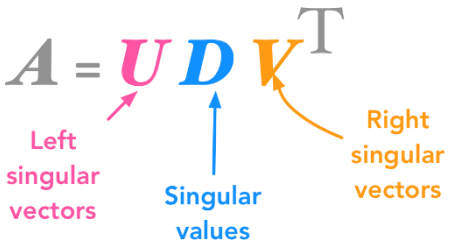
- **Learning objectives:**
  - Understand advanced matrix properties relevant to navigation
  - Master the Singular Value Decomposition (SVD) technique
  - Apply these concepts to navigation problems
  - Recognize when to use different matrix decompositions

- **Key Topics Covered:**
  - Review of fundamental matrix operations and their geometric meaning
  - Eigenvalues, eigenvectors, and matrix decompositions
  - Deep dive into SVD theory and applications
  - Introduction to Jacobian matrices for navigation systems

# Outline - Matrix Fundamentals, Differential

- Review of Basic Matrix Operations
  - Addition, multiplication, transpose, inverse
  - Determinant and special matrices
  - Navigation applications
- Advanced Matrix Properties
  - Eigenvalues and eigenvectors
  - Positive definiteness
  - Matrix condition and stability
- Matrix Decompositions Overview
  - Types of decompositions used in navigation
  - Selecting appropriate decomposition methods
- Singular Value Decomposition (SVD)
  - Theory and intuition
  - Applications to linear systems
  - Solving navigation problems
- Differentiation and Jacobian Matrices
  - Review of differentiation
  - Introduction to Jacobian matrices
  - Navigation applications



# Matrix Mathematics in Autonomous

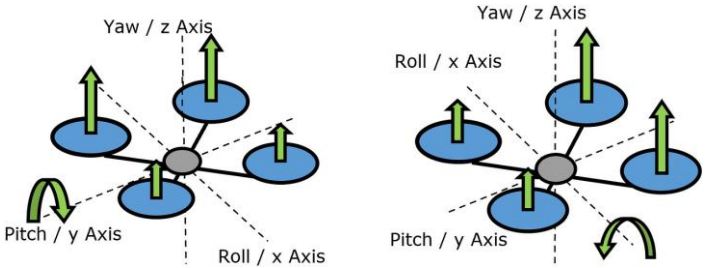



Figure: Matrix computations enable real-time navigation in autonomous systems

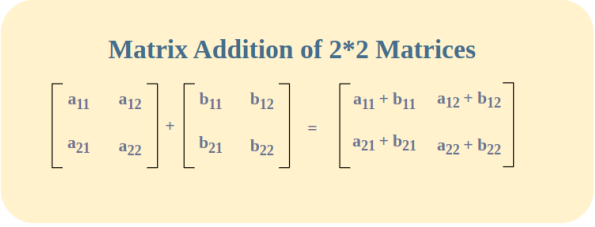
# Basic Matrix Operations Review - I: Addition &

**Matrix Addition & Subtraction**
For matrices of same dimensions:

$$\mathbf{A} + \mathbf{B} = [a_{ij} + b_{ij}] \quad (1)$$
$$\mathbf{A} - \mathbf{B} = [a_{ij} - b_{ij}] \quad (2)$$

**Example:**

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix} \quad (3)$$




**Properties:**

- Commutative: $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$
- Associative: $(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C})$
- Identity: $\mathbf{A} + \mathbf{0} = \mathbf{A}$
- Additive inverse: $\mathbf{A} + (-\mathbf{A}) = \mathbf{0}$

**Navigation application:**

- Combining multiple coordinate offsets
- Averaging multiple position estimates
- Combining error terms in navigation filters
- Adding displacement vectors in path planning

# Basic Matrix Operations Review - II:

**Matrix Multiplication**

For $\mathbf{A}_{m \times n}$ and $\mathbf{B}_{n \times p}$:

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik}\, b_{kj} \tag{4}$$

**Example:**

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \tag{5}$$

$$= \begin{bmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{bmatrix} \tag{6}$$

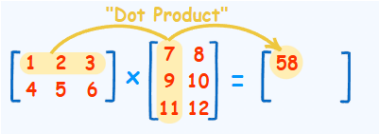
$$= \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix} \tag{7}$$

**Properties:**

- Not commutative: $\mathbf{AB} \neq \mathbf{BA}$ (generally)
- Associative: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
- Distributive: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$
- Identity: $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$

**Navigation application:**

- Applying coordinate transformations
- Converting vectors between reference frames
- Sequential rotations in 3D space
- State transition in navigation algorithms



# Basic Matrix Operations Review - III:

**Matrix Transpose**
Flips a matrix over its diagonal:

$$\left(\mathbf{A}^{\mathrm{T}}\right)_{\mathrm{ij}} = a_{\mathrm{ji}} \tag{8}$$

**Example:**

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} \tag{9}$$

**Properties:**

$$\left(\mathbf{A}^T\right)^T = \mathbf{A} \tag{10}$$

$$(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T \tag{11}$$

$$(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T \tag{12}$$

**Special cases:**

- **Symmetric matrix**: $\mathbf{A}^{\mathbf{T}} = \mathbf{A}$
  — Example: Covariance matrices
  — $\mathbf{P} = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix}$
- **Skew-symmetric matrix**: $\mathbf{A}^{\boldsymbol{T}} = -\mathbf{A}$
  — Example: Cross-product operator

$$[\vec{\omega}]_\times = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

**Navigation application:**

- Converting between inertial and body coordinates
- Efficient attitude representation
- Error propagation calculations

# Basic Matrix Operations Review - IV:

**Matrix Inverse**

For square matrix **A**, if it exists:

$$\mathbf{A}\mathbf{A}^{-1}=\mathbf{A}^{-1}\mathbf{A}=\mathbf{I} \tag{13}$$

**Properties:**

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A} \tag{14}$$

$$\left(\mathbf{A}^{\mathrm{T}}\right)^{-1} = (\mathbf{A}^{-1})^{\mathrm{T}} \tag{15}$$

$$(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1} \tag{16}$$

**2×2 Matrix Inverse Formula:**

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \tag{17}$$

**Example Calculation:**

Find the inverse of $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$

Step 1: Calculate determinant

$$det(\mathbf{A}) = 2 \times 3 - 1 \times 1 = 5 \tag{18}$$

Step 2: Apply formula

$$\mathbf{A}^{-1} = \frac{1}{5}\begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix} \tag{19}$$

$$= \begin{bmatrix} 0.6 & -0.2 \\ -0.2 & 0.4 \end{bmatrix} \tag{20}$$

**Navigation application:**

- Solving systems of linear equations
- Reversing coordinate transformations
- Computing least-squares solutions in positioning

# Matrix Calculations and Properties - V:

**Determinant**
For a $2 \times 2$ matrix:

$$det\begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc \quad (21)$$

**Example:**

$$det\begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} = 4 \times 3 - 2 \times 1 = 10 \quad (22)$$

**Properties:**

$$det(\mathbf{AB}) = det(\mathbf{A})det(\mathbf{B}) \quad (23)$$

$$det(\mathbf{A}^T) = det(\mathbf{A}) \quad (24)$$

$$det(\mathbf{A}^{-1}) = \frac{1}{det(\mathbf{A})} \quad (25)$$

**Physical meaning in 3D transformations:**

- $|\, det(\mathbf{A})|$: Volume scaling factor
- $det(\mathbf{A}) = 0$: Non-invertible (singular)

**Examples in navigation:**

- Rotation matrices: $det(\mathbf{R}) = 1$
- Reflection matrices: $det(\mathbf{R}) = -1$
- Used to check validity of transformation matrices

**Navigation application:**

- Testing if a matrix is singular before inversion

# Special Matrices in

**Orthogonal Matrices**

A square matrix $\mathbf{Q}$ is orthogonal if:

$$\mathbf{Q}^{\mathrm{T}}\mathbf{Q} = \mathbf{Q}\mathbf{Q}^{\mathrm{T}} = \mathbf{I} \tag{26}$$

**Key properties:**

- $\mathbf{Q}^{-1} = \mathbf{Q}^{T}$
- $det(\mathbf{Q}) = \pm 1$
- Preserves lengths and angles
- Columns form orthonormal basis

**Example:** 2D rotation matrix

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \tag{27}$$

**Navigation applications:**

- Attitude representation
- Direction Cosine Matrices (DCM)
- Coordinate frame transformations

**Symmetric Matrices**

A matrix $\mathbf{S}$ is symmetric if:

$$\mathbf{S}^{T} = \mathbf{S} \tag{28}$$

**Key properties:**

- All eigenvalues are real
- Eigenvectors from different eigenvalues are orthogonal
- Can be diagonalized by orthogonal matrix

**Example:** Covariance matrix

$$\mathbf{P} = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix} \tag{29}$$

**Navigation applications:**

- Position error representation
- Inertia tensors for spacecraft
- Kalman filter covariance matrices

# Advanced Matrix

### Review from Coordinate Transformations

- Rotation matrices are orthogonal: $\mathbf{R}\mathbf{R}^T = \mathbf{I}$
- Their determinant equals 1: $det(\mathbf{R}) = 1$
- Their inverse equals their transpose: $\mathbf{R}^{-1} = \mathbf{R}^T$

### Eigenvalues and Eigenvectors

- Definition: For a square matrix $\mathbf{A}$, a non-zero vector $\mathbf{v}$ is an eigenvector with eigenvalue $\lambda$ if:

$$\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v} \tag{30}$$

- Solve by finding roots of characteristic equation:

$$det(\mathbf{A} - \lambda\mathbf{I}) = 0 \tag{31}$$

- Physical meaning: Directions that are only scaled (not rotated) by the transformation

### Key Properties

- A real $n \times n$ matrix has at most $n$ eigenvalues
- Some may be complex numbers
- For a symmetric matrix, all eigenvalues are real
- For a rotation matrix, eigenvalues have magnitude 1

### Applications in Navigation

- System stability analysis
- Principal component analysis for sensor data
- Modal analysis of dynamic systems
- Error ellipsoid visualization
- Determining principal axes of inertia
- Optimizing computational efficiency

# Eigenvalue Calculation:

**Example 1: 2×2 Symmetric Matrix**
Consider the matrix:

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \tag{32}$$

**Step 1:** Form the characteristic equation

$$det(\mathbf{A} - \lambda \mathbf{I}) = \mathbf{0} \tag{33}$$

$$det \begin{bmatrix} 3-\lambda & 1 \\ 1 & 3-\lambda \end{bmatrix} = 0 \tag{34}$$

$$(3-\lambda)^2 - 1 = 0 \tag{35}$$

$$(3-\lambda)^2 = 1 \tag{36}$$

$$3 - \lambda = \pm 1 \tag{37}$$

**Step 2:** Solve for eigenvalues

$$3 - \lambda = 1 \; or \; 3 - \lambda = -1 \tag{38}$$

$$\lambda = 2 \; or \; \lambda = 4 \tag{39}$$

**Step 3**: Find eigenvectors for $\lambda_1 = 2$

$$(\mathbf{A} - 4\mathbf{I})\mathbf{v}_1 = \mathbf{0} \tag{40}$$

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{41}$$

This gives us the equation $v_{11} + v_{12} = 0$, so $v_{12} = -v_{11}$

Taking $v_{11} = 1$, we get $\mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

**Step 4**: Find eigenvectors for $\lambda_2 = 4$

$$(\mathbf{A} - 4\mathbf{I})v_2 = 0 \tag{42}$$

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{43}$$

This gives us $-v_{21} + v_{22} = 0$ ,so $v_{22} = v_{21}$

Taking $v_{21} = 1$, we get $\mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

**Verification**: $\mathbf{A}\mathbf{v}_1 = 2\mathbf{v}_1 \; and \; \mathbf{A}\mathbf{v}_2 = 4\mathbf{v}_2$

# Eigenvalue Calculation:

**Example 2: Rotation Matrix**
Consider a 2D rotation matrix for 90° rotation:

$$\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \tag{44}$$

**Step 1:** Form the characteristic equation

$$det(\mathbf{R} - \lambda \mathbf{I}) = \mathbf{0} \tag{45}$$

$$det \begin{bmatrix} -\lambda & -1 \\ 1 & -\lambda \end{bmatrix} = 0 \tag{46}$$

$$\lambda^2 + 1 = 0 \tag{47}$$

**Step 2:** Solve for eigenvalues

$$\lambda^2 = -1 \tag{48}$$

$$\lambda = \pm i \tag{49}$$

The eigenvalues are $\lambda_1 = i \; and \; \lambda_2 = -i$

**Step 3:** Find eigenvectors for $\lambda_2 = i$

$$(\mathbf{R} - i\mathbf{I})\mathbf{v}_1 = 0 \tag{50}$$

$$\begin{bmatrix} -i & -1 \\ 1 & -i \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{51}$$

This gives us $-iv_{11} - v_{12} = 0, v_{12} = -iv_{11}$

Taking $v_{11} = 1$, we get $\mathbf{v}_1 = \begin{bmatrix} 1 \\ -i \end{bmatrix}$

**Step 4:** Find eigenvectors for $\lambda_2 = -i$

$$(\mathbf{R} + i\mathbf{I})\mathbf{v}_2 = \mathbf{0} \tag{52}$$

$$\begin{bmatrix} i & -1 \\ 1 & i \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{53}$$

This gives us $iv_{21} - v_{22} = 0, \; so \; v_{22} = iv_{21}$

Taking $v_{21} = 1$, we get $\mathbf{v}_2 = \begin{bmatrix} 1 \\ i \end{bmatrix}$

**Physical meaning:** A 90° rotation has no real eigenvectors!

# Eigenvalue Calculation: Example 3

**Example 3: Navigation Application** Consider a covariance matrix from GNSS measurements:

$$\mathbf{P} = \begin{bmatrix} 25 & 15 \\ 15 & 16 \end{bmatrix} \text{m}^2 \tag{54}$$

**Step 1:** Form the characteristic equation

$$det(\mathbf{P} - \lambda \mathbf{I}) = \mathbf{0} \tag{55}$$

$$det \begin{bmatrix} 25-\lambda & 15 \\ 15 & 16-\lambda \end{bmatrix} = 0 \tag{56}$$

$$(25-\lambda)(16-\lambda) - 15^2 = 0 \tag{57}$$

$$400 - 25\lambda - 16\lambda + \lambda^2 - 225 = 0 \tag{58}$$

$$\lambda^2 - 41\lambda + 175 = 0 \tag{59}$$

**Step 2:** Solve for eigenvalues using the quadratic formula

$$\lambda = \frac{41 \pm \sqrt{41^2 - 4 \times 175}}{2} \tag{60}$$

$$= \frac{41 \pm \sqrt{1681 - 700}}{2} \tag{61}$$

$$= \frac{41 \pm \sqrt{981}}{2} \tag{62}$$

$$= \frac{41 \pm 31.32}{2} \tag{63}$$

Therefore:

$$\lambda_1 \approx \frac{41 + 31.32}{2} \approx 36.1 \text{ m}^2 \tag{64}$$

$$\lambda_2 \approx \frac{41 - 31.32}{2} \approx 4.9 \text{ m}^2 \tag{65}$$

# Eigenvalue Calculation for Scaling Matrix

**Matrix from Visual Example**

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \tag{66}$$

**Step 1:** Form the characteristic equation

$$det(\mathbf{A} - \lambda \mathbf{I}) = \mathbf{0} \tag{67}$$

$$det \begin{bmatrix} 2-\lambda & 0 \\ 0 & 0.5-\lambda \end{bmatrix} = 0 \tag{68}$$

For a diagonal matrix, the determinant is the product of the diagonal elements:

$$(2-\lambda)(0.5-\lambda) = 0 \tag{69}$$

$$\lambda^2 - 2.5\lambda + 1 = 0 \tag{70}$$

**Step 2:** Solve for eigenvalues
The equation factorizes directly:

$$(2-\lambda)(0.5-\lambda) = 0 \tag{71}$$

$$\lambda = 2 \text{ or } \lambda = 0.5 \tag{72}$$

# Eigenvalue Calculation for Scaling Matrix

**Step 3:** Find eigenvector for $\lambda_1 = 2$

$$(\mathbf{A} - 2\mathbf{I})\mathbf{v}_1 = \mathbf{0} \tag{73}$$

$$\begin{bmatrix} 0 & 0 \\ 0 & -1.5 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{74}$$

This simplifies to:

$$0 \cdot v_{11} + 0 \cdot v_{12} = 0 \tag{75}$$

$$0 \cdot v_{11} + (-1.5) \cdot v_{12} = 0 \tag{76}$$

The second equation gives us: $-1.5v_{12} = 0$, which means $v_{12} = 0$

The equation is satisfied for any value of $v_{11}$

Taking $v_{11} = 1$ to normalize:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \tag{77}$$

**Verification for $\lambda_1 = 2$:**

$$\mathbf{A}\mathbf{v}_1 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 2\mathbf{v}_1 \tag{78}$$

# Eigenvalue Calculation for Scaling Matrix

**Step 4:** Find eigenvector for $\lambda_2 = 0.5$

$$(\mathbf{A} - 0.5\mathbf{I})\mathbf{v}_2 = \mathbf{0} \tag{79}$$

$$\begin{bmatrix} 1.5 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{80}$$

This simplifies to:

$$1.5 \cdot v_{21} + 0 \cdot v_{22} = 0 \tag{81}$$

$$0 \cdot v_{21} + 0 \cdot v_{22} = 0 \tag{82}$$

The first equation gives us: $1.5v_{21} = 0$, which means $v_{21} = 0$

The second equation is satisfied for any value of $v_{22}$

Taking $v_{22} = 1$ to normalize:

$$\mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{83}$$

**Verification for $\lambda_2 = 0.5$:**

$$\mathbf{A}\mathbf{v}_2 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} = 0.5 \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0.5\mathbf{v}_2 \tag{84}$$

# Eigenvalue Calculation: Summary and Key

**Results Summary:**

- Eigenvalues: $\lambda_1 = 2 \; and \; \lambda_2 = 0.5$
- Eigenvectors: $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \; and \; \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

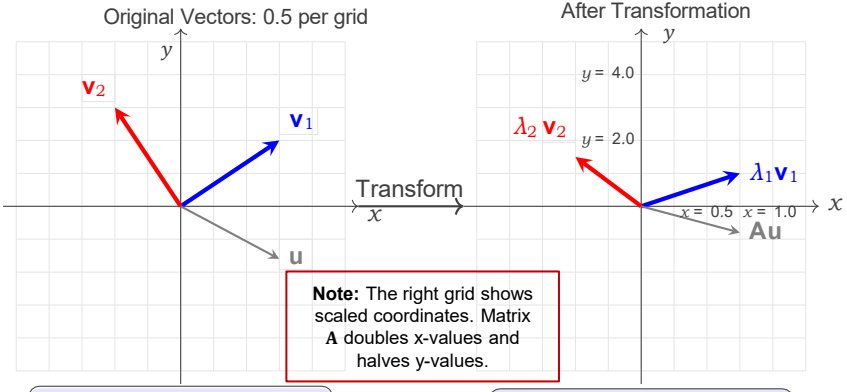
**Key Observations:**

- For this diagonal matrix, the eigenvectors are simply the standard basis vectors
- The eigenvalues are the diagonal elements of the matrix
- Each eigenvector is stretched/compressed only along its axis:
  - $\mathbf{v}_1$ (along x-axis) is stretched by factor 2
  - $\mathbf{v}_2$ (along y-axis) is compressed by factor 0.5
- This explains the visual transformation
- Any matrix can be decomposed into components along these eigenvectors
- The transformation's effect is to scale each component by its corresponding eigenvalue

# Eigenvectors and Eigenvalues: Visual

**Eigenvector Equation:** $\mathbf{Av} = \lambda\mathbf{v}$

**Key Property:** Eigenvectors maintain their direction under transformation




**Note:** The right grid shows scaled coordinates. Matrix $\mathbf{A}$ doubles x-values and halves y-values.

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix}$$

$\lambda_1 = 2$ (blue eigenvector)
$\lambda_2 = 0.5$ (red eigenvector)

# Connecting Calculations to Visualization

**Vectors from Our Visual Example**
In our visualization, we used:

$$\mathbf{v_1} = \begin{bmatrix} 1.5 \\ 1 \end{bmatrix} \quad (85)$$

$$\mathbf{v}_2 = \begin{bmatrix} -1 \\ 1.5 \end{bmatrix} \quad (86)$$

$$\mathbf{u} = \begin{bmatrix} 1.5 \\ -0.8 \end{bmatrix} \quad (87)$$

**Decomposition into Eigenvectors**
Any vector can be written as a linear combination of eigenvectors:

$$\mathbf{v} = c_1\mathbf{e}_1 + c_2\mathbf{e}_2 \quad (88)$$

Where $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ are our calculated eigenvectors.

**Expressing Our Vectors**

$$\mathbf{v}_1 = 1.5\mathbf{e}_1 + 1\mathbf{e}_2 \quad (89)$$

$$\mathbf{v}_2 = -1\mathbf{e}_1 + 1.5\mathbf{e}_2 \quad (90)$$

$$\mathbf{u} = 1.5\mathbf{e}_1 - 0.8\mathbf{e}_2 \quad (91)$$

**Transformation Property**
For any vector $\mathbf{v} = c_1\mathbf{e}_1 + c_2\mathbf{e}_2$ :

$$\mathbf{A}\mathbf{v} = \mathbf{A}(c_1\mathbf{e}_1 + c_2\mathbf{e}_2) \quad (92)$$

$$= c_1\mathbf{A}\mathbf{e}_1 + c_2\mathbf{A}\mathbf{e}_2 \quad (93)$$

$$= c_1\lambda_1\mathbf{e}_1 + c_2\lambda_2\mathbf{e}_2 \quad (94)$$

$$= 2c_1\mathbf{e}_1 + 0.5c_2\mathbf{e}_2 \quad (95)$$

# Connecting Calculations to Visualization

**Transforming Vector $\mathbf{v}_1$**

$$\mathbf{A}\mathbf{v}_1 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1.5 \\ 1 \end{bmatrix} \quad (96)$$

$$= \begin{bmatrix} 2 \times 1.5 \\ 0.5 \times 1 \end{bmatrix} \quad (97)$$

$$= \begin{bmatrix} 3 \\ 0.5 \end{bmatrix} \quad (98)$$

Alternatively, using eigenvector decomposition:

$$\mathbf{A}\mathbf{v}_1 = \mathbf{A}(1.5\mathbf{e}_1 + 1\mathbf{e}_2) \quad (99)$$

$$= 1.5(2\mathbf{e}_1) + 1(0.5\mathbf{e}_2) \quad (100)$$

$$= 3\mathbf{e}_1 + 0.5\mathbf{e}_2 \quad (101)$$

$$= \begin{bmatrix} 3 \\ 0.5 \end{bmatrix} \quad (102)$$

**Transforming Vector $\mathbf{v}_2$**

$$\mathbf{A}\mathbf{v}_2 = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} -1 \\ 1.5 \end{bmatrix} \quad (103)$$

$$= \begin{bmatrix} 2 \times (-1) \\ 0.5 \times 1.5 \end{bmatrix} \quad (104)$$

$$= \begin{bmatrix} -2 \\ 0.75 \end{bmatrix} \quad (105)$$

**Transforming Vector u**

$$\mathbf{A}\mathbf{u} = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1.5 \\ -0.8 \end{bmatrix} \quad (106)$$

$$= \begin{bmatrix} 2 \times 1.5 \\ 0.5 \times (-0.8) \end{bmatrix} \quad (107)$$

$$= \begin{bmatrix} 3 \\ -0.4 \end{bmatrix} \quad (108)$$

# In class practice (solution in class tutorials)

## 1 Exercise 1: Eigenvalue Decomposition

### 1.1 Problem Statement

**Navigation Scenario**

A GPS/INS integrated navigation system is operating in an urban environment. The position error covariance matrix in the local East-North-Up (ENU) coordinate frame has been estimated as:

$$\mathbf{P} = \begin{bmatrix} 9 & 6 \\ 6 & 16 \end{bmatrix} \text{ m}^2$$

This 2×2 matrix represents the horizontal position uncertainty (East and North components only, assuming vertical component is handled separately).

#### 1.1.1 Required Tasks

1. Calculate all eigenvalues of the covariance matrix $\mathbf{P}$
2. Determine the corresponding normalized eigenvectors
3. Verify the eigenvalue-eigenvector relationships
4. Provide physical interpretation of the results in navigation context
5. Construct and analyze the $2\sigma$ error ellipse
6. Discuss implications for navigation system performance

# Eigenvalues and Eigenvectors: Why They Matter in



### Theoretical Importance

- Fundamental property of linear transformations
- Simplify complex matrix operations
- Enable matrix diagonalization
- Provide insight into system stability
- Characterize key directions in data

### Mathematical Applications

- Principal Component Analysis (PCA)
- Solving coupled differential equations
- Optimizing computational efficiency
- Analyzing dynamic systems

### Navigation Applications

- **Error Ellipses:** Semi-major and semi-minor axes of position error ellipses in GNSS
- **Inertial Navigation:** Principal axes of inertia in vehicle dynamics
- **Sensor Fusion:** Optimally combining data from multiple sensors
- **Attitude Determination:** Finding optimal rotation from measurements
- **Kalman Filtering:** Understanding error propagation in state estimation

**Key Takeaway:** Eigenvalues and eigenvectors provide critical insight into transformations and error characteristics, helping us design robust navigation systems.

# Advanced Matrix

**Positive Definite Matrices**

- A symmetric matrix **P** is positive definite if:

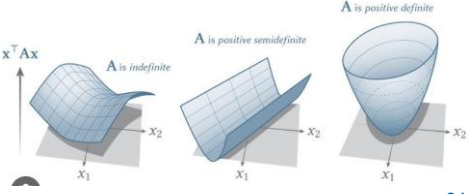
$$\mathbf{x}^T\mathbf{P}\mathbf{x} > 0 \quad \text{for all non-zero vectors } \mathbf{x} \tag{109}$$

- Equivalent conditions:
  - All eigenvalues are positive
  - All leading principal minors are positive
  - $\mathbf{P} = \mathbf{L}\mathbf{L}^T$ for some non-singular $\mathbf{L}$ (Cholesky decomposition)
- A matrix is positive semi-definite if:

$$\mathbf{x}^T\mathbf{P}\mathbf{x} \geq 0 \text{ for all vectors } \mathbf{x} \tag{110}$$

**Navigation Applications**

- Covariance matrices in estimation
  - Must be positive definite
  - Ensures physical consistency
  - Determines uncertainty ellipsoids
- Information matrices in optimization
  - Used in least squares estimation
  - Inverse of covariance matrix
- Weighting matrices in measurement



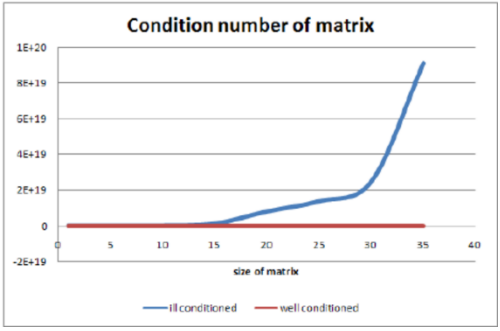
# Advanced Matrix Properties

### Matrix Condition Number

- Measures sensitivity to numerical errors:

$$\kappa(\mathbf{A}) = \| \mathbf{A} \| \cdot \| \mathbf{A}^{-1} \| = \frac{\sigma_{max}}{\sigma_{min}} \qquad (111)$$

where $\sigma\max$ and $\sigma\min$ are the largest and smallest singular values




(112)

### Interpretation

- $\kappa \approx 1$: Well-conditioned (stable)
- $\kappa \gg 1$: Ill-conditioned (unstable)
- $\kappa = \infty$: Singular (non-invertible)

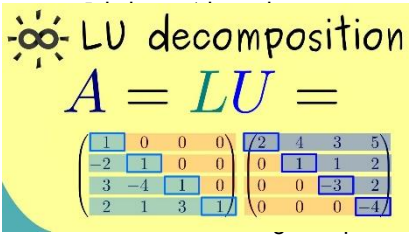
### Navigation Implications

- Poor satellite geometry leads to high condition numbers
  - Related to DOP (Dilution of Precision)
  - Affects accuracy of position solutions
- Guides selection of numerical methods
  - When to use regularization
  - Which decomposition to select
  - How much to trust solutions

# Matrix Decompositions

**Common Matrix Decompositions**

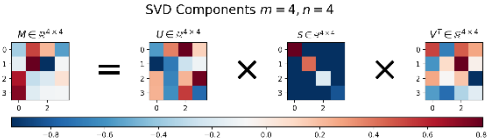
- **Eigendecomposition**
  - $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1}$
  - For diagonalizable matrices
  - $\mathbf{Q}$ contains eigenvectors
  - $\mathbf{\Lambda}$ is diagonal with eigenvalues
  - Useful for understanding system dynamics
- **Singular Value Decomposition**
  - $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
  - Works for any matrix (even rectangular)
  - $\mathbf{U}, \mathbf{V}$ are orthogonal
  - $\mathbf{\Sigma}$ contains singular values
  - Best for solving linear systems
- **QR Decomposition**
  - $\mathbf{A} = \mathbf{QR}$
  - $\mathbf{Q}$ is orthogonal
  - $\mathbf{R}$ is upper triangular
  - Efficient for sequential processing
  - Used in real-time applications
- **Cholesky Decomposition**
  - $\mathbf{A} = \mathbf{L}\mathbf{L}^T$
  - For positive definite matrices only




systems

# Choosing the Right

| Decomposition | Best for | Restrictions | Navigation Use |
|---|---|---|---|
| Eigendecomposition | System analysis | Square matrices | Controller design |
| SVD | Linear systems | None | GNSS positioning |
| QR | Sequential processing | None | Real-time filtering |
| Cholesky | Efficient solving | Positive definite | Kalman filtering |




**Selection Criteria**

- **Computational efficiency**
  - Cholesky: $\frac{1}{3}n^3$ flops (floating-point operations per second)
  - QR: $2n^2m - \frac{2}{3}n^3$ flops for $m \times n$ matrix
  - SVD: $4n^2m + 8nm^2 + 9m^3$ flops
- **Numerical stability** - SVD is most stable for ill-conditioned systems
- **Problem structure** - Can be exploited for efficiency
- **Required information** - Some decompositions reveal specific properties (rank, null space)

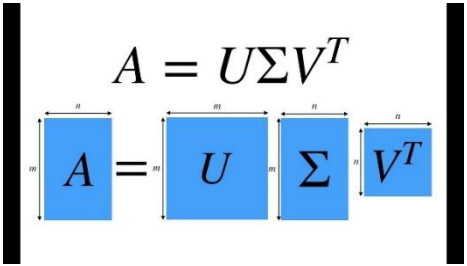
# Singular Value Decomposition - I: The

**The SVD Theorem**

- Any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ can be factored as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \qquad (113)$$

- Where:
  - $\mathbf{U} \in \mathbb{R}^{m \times m}$ is orthogonal $\mathbf{U}^T\mathbf{U} = \mathbf{I}$
  - $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is diagonal with non-negative entries
  - $\mathbf{V} \in \mathbb{R}^{n \times n}$ is orthogonal $\mathbf{V}^T\mathbf{V} = \mathbf{I}$




- Diagonal entries σi of $\mathbf{\Sigma}$ are the singular values of $\mathbf{A}$
- Singular values are arranged in descending order: $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$
- Number of non-zero singular values equals the rank of $\mathbf{A}$
- The columns of $\mathbf{U}$ are called left singular vectors
- The columns of $\mathbf{V}$ are called right singular vectors

# Singular Value Decomposition - II: Matrix



**Matrix Visualization**

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{114}$$

Expanded form:

$$\mathbf{A} = \begin{bmatrix} | & | & \cdots & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_m \\ | & | & \cdots & | \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n \end{bmatrix} \begin{bmatrix} - & - & \mathbf{v}_1^T & - & - \\ - & - & \mathbf{v}_2^T & - & - \\ & \vdots & \vdots & \vdots & \\ - & - & \mathbf{v}_n^T & - & - \end{bmatrix} \tag{115}$$

**Alternative Expression**

$$\mathbf{A} = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^T \tag{116}$$

Where $r$ is the rank of matrix $\mathbf{A}$.

This expresses the matrix as a sum of rank-1 matrices, weighted by singular values.

# SVD Example Calculation -

**Example: Find the SVD of matrix A**

$$\mathbf{A} = \begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix} \quad (123)$$

**The rank of A is 2**

**Step 1:** Calculate $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 2 & 1 & 3 \\ 2 & 3 & 1 \end{bmatrix}\begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix} \quad (124)$$

$$= \begin{bmatrix} 4+1+9 & 4+3+3 \\ 4+3+3 & 4+9+1 \end{bmatrix} \quad (125)$$

$$= \begin{bmatrix} 14 & 10 \\ 10 & 14 \end{bmatrix} \quad (126)$$

Similarly:

$$\mathbf{A}\mathbf{A}^T = \begin{bmatrix} 8 & 8 & 8 \\ 8 & 10 & 6 \\ 8 & 6 & 10 \end{bmatrix} \quad (127)$$

**Step 2:** Find eigenvalues of $\mathbf{A}^T\mathbf{A}$

$$det(\mathbf{A}^T\mathbf{A} - \lambda\mathbf{I}) = \mathbf{0} \quad (128)$$

$$det\begin{bmatrix} 14-\lambda & 10 \\ 10 & 14-\lambda \end{bmatrix} = 0 \quad (129)$$

$$(14-\lambda)^2 - 100 = 0 \quad (130)$$

$$\lambda^2 - 28\lambda + 96 = 0 \quad (131)$$

$$(\lambda - 24)(\lambda - 4) = 0 \quad (132)$$

So $\lambda_1 = 24$ and $\lambda_2 = 4$

**Step 3:** Calculate singular values

$$\sigma_i = \sqrt{\lambda_i} \quad (133)$$

Therefore:

$$\sigma_1 = \sqrt{24} = 2\sqrt{6} \quad (134)$$

$$\sigma_2 = \sqrt{4} = 2 \quad (135)$$

# Why the rank of matrix A is 2?

$$A = \begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix}$$

The rank of matrix A is 2 because:

1. **A is a 3×2 matrix**, so the maximum possible rank is min(3,2) = 2.
2. **The two columns are linearly independent**. We can verify this by checking if one column is a scalar multiple of the other:
   - Column 1: $[2, 1, 3]^T$
   - Column 2: $[2, 3, 1]^T$

   There's no scalar k such that $[2, 1, 3]^T = k[2, 3, 1]^T$, so the columns are linearly independent.
3. **We can also verify by row reduction**:
   $$\begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 \\ 0 & 2 \\ 0 & -2 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 \\ 0 & 2 \\ 0 & 0 \end{bmatrix}$$

   The row-reduced form has 2 non-zero rows, confirming rank = 2.

# SVD Example Calculation -

**Step 4:** Find right singular vectors (eigenvectors of $\mathbf{A}^T\mathbf{A}$)

For $\lambda_1 = 24$

$$(\mathbf{A}^T\mathbf{A} - 24\mathbf{I})\mathbf{v}_1 = \mathbf{0} \quad (136)$$

$$\begin{bmatrix} -10 & 10 \\ 10 & -10 \end{bmatrix}\mathbf{v}_1 = 0 \quad (137)$$

This gives us: $-10v_{11} + 10v_{12} = 0$ ,so $v_{11} = v_{12}$

Normalizing: $\mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

For $\lambda_2 = 4$:

$$(\mathbf{A}^T\mathbf{A} - 4\mathbf{I})\mathbf{v}_2 = \mathbf{0} \quad (138)$$

$$\begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}\mathbf{v}_2 = 0 \quad (139)$$

This gives us: $v_{21} = -v_{22}$

Normalizing: $\mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$

**Step 5:** Calculate left singular vectors Method 1: Find eigenvectors of $\mathbf{A}\mathbf{A}^T$) same as $\mathbf{A}^T\mathbf{A}$ in this case)

$$\boldsymbol{u}_1 = \frac{1}{\sqrt{3}}\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \boldsymbol{u}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix} \textit{and } \boldsymbol{u}_3 = \frac{1}{\sqrt{2}}\begin{bmatrix} -\frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} \end{bmatrix}$$

The unit vector $\boldsymbol{u}_3$ is computed to be orthogonal to both $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$.

Process：

$$\mathbf{u}_3 = \begin{bmatrix} \mathbf{u}_{31} \\ \mathbf{u}_{32} \\ \mathbf{u}_{33} \end{bmatrix} \quad \begin{matrix} \mathbf{u}_3^T\mathbf{u}_1 = 0 \\ \mathbf{u}_3^T\mathbf{u}_2 = 0 \\ ||\mathbf{u}_3|| = 1 \end{matrix} \Rightarrow \begin{cases} \mathbf{u}_{31} + \mathbf{u}_{32} + \mathbf{u}_{33} = 0 \\ -\mathbf{u}_{32} + \mathbf{u}_{33} = 0 \\ \mathbf{u}_{31}^2 + \mathbf{u}_{32}^2 + \mathbf{u}_{33}^2 = 1 \end{cases}$$

$$\Rightarrow \quad \mathbf{u}_3 = \begin{bmatrix} \mathbf{u}_{31} \\ \mathbf{u}_{32} \\ \mathbf{u}_{33} \end{bmatrix} = \begin{bmatrix} -\frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{6}} \end{bmatrix}$$

# SVD Example Calculation -

Final

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$$

$$\boldsymbol{U} = \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & -\frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \end{bmatrix}$$

$$\boldsymbol{\Sigma} = \begin{bmatrix} 2\sqrt{6} & 0 \\ 0 & 2 \\ 0 & 0 \end{bmatrix}$$

$$\mathbf{V}^T = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}$$

**Method 2**: Using the formula $\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i$

$$\mathbf{u_1} = \frac{1}{2\sqrt{6}}\mathbf{A}\mathbf{v}_1$$

$$= \frac{1}{2\sqrt{6}} \begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$= \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Similar calculation confirms $\mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$

Final $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$

$$\mathbf{A} = 2\sqrt{6} \begin{bmatrix} \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix} + 2 \begin{bmatrix} 0 \\ -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}$$

# In class practice for SVD decomposition (solution in class tutorials)



## 2 Exercise 2: Singular Value Decomposition

### 2.1 Problem Statement

**Navigation Scenario**

A navigation system collects measurement data from multiple sensors. The measurement matrix $\mathbf{A}$ represents the relationship between sensor readings and position estimates in a simplified 2D navigation problem:

$$\mathbf{A} = \begin{bmatrix} 2 & 2 \\ 1 & 3 \\ 3 & 1 \end{bmatrix}$$

where:

- Each row represents a different sensor measurement
- The columns correspond to East and North position components
- The rank of $\mathbf{A}$ is 2

#### 2.1.1 Required Tasks

1. Calculate $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$
2. Find eigenvalues of $\mathbf{A}^T\mathbf{A}$ to determine singular values
3. Calculate the singular values $\sigma_i$
4. Find the right singular vectors (eigenvectors of $\mathbf{A}^T\mathbf{A}$)
5. Find the left singular vectors (eigenvectors of $\mathbf{A}\mathbf{A}^T$)
6. Construct the complete SVD: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
7. Verify the decomposition

# SVD Example Calculation -

**Example: Find the SVD of matrix A**

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix} \quad (123)$$

The rank of $\boldsymbol{A}$ is 1

**Step 1:** Calculate $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}\begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix} \quad (124)$$

$$= \begin{bmatrix} 1+4+9 & 2+8+18 \\ 2+8+18 & 4+16+36 \end{bmatrix} \quad (125)$$

$$= \begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix} \quad (126)$$

Similarly:

$$\mathbf{A}\mathbf{A}^T = \begin{bmatrix} 5 & 10 & 15 \\ 10 & 20 & 30 \\ 15 & 30 & 45 \end{bmatrix} \quad (127)$$

**Step 2:** Find eigenvalues of $\mathbf{A}^T\mathbf{A}$

$$det(\mathbf{A}^T\mathbf{A} - \lambda\mathbf{I}) = \mathbf{0} \quad (128)$$

$$det\begin{bmatrix} 14-\lambda & 28 \\ 28 & 56-\lambda \end{bmatrix} = 0 \quad (129)$$

$$(14-\lambda)(56-\lambda) - 784 = 0 \quad (130)$$

$$\lambda^2 - 70\lambda = 0 \quad (131)$$

$$(\lambda - 70)\lambda = 0 \quad (132)$$

So $\lambda_1 = 70$ and $\lambda_2 = 0$

**Step 3:** Calculate singular values

$$\sigma_i = \sqrt{\lambda_i} \quad (133)$$

Therefore:

$$\sigma_1 = \sqrt{70} \quad (134)$$

$$\sigma_2 = 0 \quad (135)$$

# Why the rank of matrix A is 1?

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$$

The rank of matrix A is 1 because:

1. **The columns are linearly dependent**. Notice that:
   - Column 2 = 2 × Column 1
   - $[2, 4, 6]^T = 2 \times [1, 2, 3]^T$
2. **The rows are also linearly dependent**:
   - Row 2 = 2 × Row 1: [2, 4] = 2 × [1, 2]
   - Row 3 = 3 × Row 1: [3, 6] = 3 × [1, 2]
3. **Row reduction would show**:
   $$\begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$
   Only one non-zero row remains, confirming rank = 1.

Since all columns (and rows) are scalar multiples of each other, the matrix has only one linearly independent column/row, making its rank 1.

# SVD Example Calculation -

**Step 4:** Find right singular vectors (eigenvectors of $\mathbf{A}^T\mathbf{A}$)

For $\lambda_1 = 70$

$$(\mathbf{A}^T\mathbf{A} - 70\mathbf{I})\mathbf{v}_1 = \mathbf{0} \quad (136)$$

$$\begin{bmatrix} -56 & 28 \\ 28 & -14 \end{bmatrix}\mathbf{v}_1 = 0 \quad (137)$$

This gives us: $2v_{11} - v_{12} = 0$ ,so $2v_{11} = v_{12}$

Normalizing: $\mathbf{v}_1 = \begin{bmatrix} \frac{1}{\sqrt{5}} \\ \frac{2}{\sqrt{5}} \end{bmatrix}$

For $\lambda_2 = 0$:

$$(\mathbf{A}^T\mathbf{A} - \mathbf{0})\mathbf{v}_2 = \mathbf{0} \quad (138)$$

$$\begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix}\mathbf{v}_2 = 0 \quad (139)$$

This gives us: $v_{21} = -2v_{22}$

Normalizing: $\mathbf{v}_2 = \begin{bmatrix} \frac{2}{\sqrt{5}} \\ -\frac{1}{\sqrt{5}} \end{bmatrix}$

**Step 5:** Calculate left singular vectors Method 1: Find eigenvectors of $\mathbf{A}\mathbf{A}^T$)same as $\mathbf{A}^T\mathbf{A}$ in this case)

$$\mathbf{u}_1 = \begin{bmatrix} \frac{1}{\sqrt{14}} \\ \frac{2}{\sqrt{14}} \\ \frac{3}{\sqrt{14}} \end{bmatrix}$$

Choose $\mathbf{u}_2 = \begin{bmatrix} \frac{2}{\sqrt{5}} \\ \frac{-1}{\sqrt{5}} \\ 0 \end{bmatrix}$ (Orthogonal to $\mathbf{u}_1$)

Notice: The answer $\mathbf{u}_2$ of is not unique.

Choose $\mathbf{u}_3 = \begin{bmatrix} \frac{3}{\sqrt{70}} \\ \frac{6}{\sqrt{70}} \\ \frac{-7}{\sqrt{70}} \end{bmatrix}$ (Orthogonal to $\mathbf{u}_1$ and $\mathbf{u}_2$ )

Notice: Since only $\sigma_1$ is non-zero and all others are zero, we only need to compute $\mathbf{v}_1$ and $\mathbf{u}_1$.

# SVD Example Calculation -

Final

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$$

$$\boldsymbol{U} = \begin{bmatrix} \frac{1}{\sqrt{14}} & \frac{2}{\sqrt{5}} & \frac{3}{\sqrt{70}} \\ \frac{2}{\sqrt{14}} & \frac{-1}{\sqrt{5}} & \frac{6}{\sqrt{70}} \\ \frac{3}{\sqrt{14}} & 0 & \frac{-7}{\sqrt{70}} \end{bmatrix}$$

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sqrt{70} & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\mathbf{V}^T = \begin{bmatrix} \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \\ \frac{2}{\sqrt{5}} & -\frac{1}{\sqrt{5}} \end{bmatrix}$$

**Method 2**: Using the formula $\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i$

$$\mathbf{u_1} = \frac{1}{\sqrt{70}}\mathbf{A}\mathbf{v}_1$$

$$= \frac{1}{\sqrt{70}}\begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}\begin{bmatrix} \frac{1}{\sqrt{5}} \\ \frac{2}{\sqrt{5}} \end{bmatrix}$$

$$= \frac{1}{\sqrt{14}}\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

So

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^T = \sqrt{70}\begin{bmatrix} \frac{1}{\sqrt{14}} \\ \frac{2}{\sqrt{14}} \\ \frac{3}{\sqrt{14}} \end{bmatrix}\begin{bmatrix} \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix}$$

# Singular Value Decomposition - III:

### Geometric Interpretation

- $\mathbf{V}^T$: Rotation in input space
- $\mathbf{\Sigma}$: Scaling along new coordinate axes
- $\mathbf{U}$: Rotation in output space

### Transformation Sequence

1. First, rotate the input space (using $\mathbf{V}^T$)
2. Then, scale along principal axes (using $\mathbf{\Sigma}$)
3. Finally, rotate in output space (using $\mathbf{U}$)

### Key Properties

- SVD reveals principal directions and stretching factors of the transformation
- SVD always exists for any matrix (unlike eigendecomposition)
- Useful for low-rank approximations:

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

- Basis for Principal Component Analysis
- Essential for pseudo-inverse calculation:

$$\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^T$$

- Helps solve least-squares problems
- Powerful tool for noise reduction in data

The singular values $\sigma_i$ indicate the importance of each principal direction in the transformation. The largest singular values correspond to the most significant components.

# Singular Value

### Compact Form of SVD

- If $\text{rank}(\mathbf{A}) = r$ ,we can use reduced form:

$$\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^T \qquad (117)$$

- Where:
  - $\mathbf{U}_r \in \mathbb{R}^{m \times r}$ contains first r columns of $\mathbf{U}$
  - $\mathbf{U}_r \in \mathbb{R}^{m \times r}$ is diagonal with non-zero singular values
  - $\mathbf{V}_r \in \mathbb{R}^{n \times r}$ contains first r columns of $\mathbf{V}$
- Matrix rank = number of non-zero singular values
- Allows low-rank approximation by keeping only the largest singular values:

$$\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T \qquad (118)$$

which is the best rank-*k* approximation to $\mathbf{A}$ in the Frobenius norm

### Advantages of Compact SVD

- Reduces storage requirements
- Improves computational efficiency
- Focuses on effective dimensions of the problem
- Reveals essential structure of the transformation
- Enables data compression

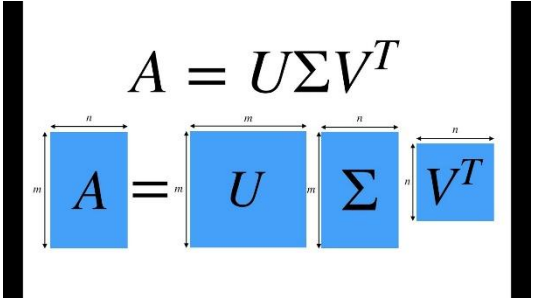
### Navigation Applications

- Removing redundant measurements
- Isolating important signal components
- Reducing sensor data size
- Filtering out noise (truncated SVD)
- Computational speedups in large systems

# Singular Value Decomposition – IV (try the

**Properties of SVD**

- **Relationship to eigendecomposition**
  - Columns of **U** are eigenvectors of $\mathbf{A}\mathbf{A}^T$
  - Columns of **V** are eigenvectors of $\mathbf{A}^T\mathbf{A}$
  - $\sigma^2$ are eigenvalues of both $\mathbf{A}^T\mathbf{A}$ *and* $\mathbf{A}\mathbf{A}^T$
  - For symmetric positive definite matrices: $\sigma_i = |\lambda_i|$




**Matrix Norms via SVD**

- **Frobenius norm**

$$\| \mathbf{A} \|_F = \sqrt{\sum_{i=1}^{r} \sigma_i^2} \qquad (119)$$

- **2-norm** (operator norm)

$$\| \mathbf{A} \|_2 = \sigma_1 \quad \text{(largest singular value)} \qquad (120)$$

- **Condition number**

$$\kappa(\mathbf{A}) = \frac{\sigma_1}{\sigma_r} \quad \text{(ratio of largest to smallest)} \qquad (121)$$

- **Nuclear norm**

$$\| \mathbf{A} \|_* = \sum_{i=1}^{r} \sigma_i \quad \text{(sum of singular values)}$$

# SVD Example Calculation -

**Example: Find the SVD of matrix A**

$$\mathbf{A} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \tag{123}$$

**Step 1:** Calculate $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \tag{124}$$

$$= \begin{bmatrix} 9+4 & 6+6 \\ 6+6 & 4+9 \end{bmatrix} \tag{125}$$

$$= \begin{bmatrix} 13 & 12 \\ 12 & 13 \end{bmatrix} \tag{126}$$

Similarly:

$$\mathbf{A}\mathbf{A}^T = \begin{bmatrix} 13 & 12 \\ 12 & 13 \end{bmatrix} \tag{127}$$

**Step 2:** Find eigenvalues of $\mathbf{A}^T\mathbf{A}$

$$det(\mathbf{A}^T\mathbf{A} - \lambda\mathbf{I}) = \mathbf{0} \tag{128}$$

$$det\begin{bmatrix} 13-\lambda & 12 \\ 12 & 13-\lambda \end{bmatrix} = 0 \tag{129}$$

$$(13-\lambda)^2 - 12^2 = 0 \tag{130}$$

$$(13-\lambda)^2 = 144 \tag{131}$$

$$13-\lambda = \pm 12 \tag{132}$$

So $\lambda_1 = 25$ and $\lambda_2 = 1$

**Step 3:** Calculate singular values

$$\sigma_i = \sqrt{\lambda_i} \tag{133}$$

Therefore:

$$\sigma_1 = \sqrt{25} = 5 \tag{134}$$

$$\sigma_2 = \sqrt{1} = 1 \tag{135}$$

# SVD Example Calculation -

**Step 4:** Find right singular vectors (eigenvectors of $\mathbf{A}^T\mathbf{A}$)

For $\lambda_1 = 25$:

$$(\mathbf{A}^T\mathbf{A} - 25\mathbf{I})\mathbf{v}_1 = \mathbf{0} \quad (136)$$

$$\begin{bmatrix} -12 & 12 \\ 12 & -12 \end{bmatrix}\mathbf{v}_1 = 0 \quad (137)$$

This gives us: $-12v_{11} + 12v_{12} = 0$ ,so $v_{11} = v_{12}$

Normalizing: $\mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

For $\lambda_2 = 1$:

$$(\mathbf{A}^T\mathbf{A} - 1\mathbf{I})\mathbf{v}_2 = \mathbf{0} \quad (138)$$

$$\begin{bmatrix} 12 & 12 \\ 12 & 12 \end{bmatrix}\mathbf{v}_2 = 0 \quad (139)$$

This gives us: $v_{21} = -v_{22}$

Normalizing: $\mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$

**Step 5:** Calculate left singular vectors Method 1: Find eigenvectors of $\mathbf{A}\mathbf{A}^T$)same as $\mathbf{A}^T\mathbf{A}$ in this case)

$$\mathrm{u}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix} and\ \boldsymbol{u}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

**Method 2**: Using the formula $\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i$

$$\mathbf{u_1} = \frac{1}{5}\mathbf{A}\mathbf{v}_1 \quad (140)$$

$$= \frac{1}{5}\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}\frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad (141)$$

$$= \frac{1}{5\sqrt{2}}\begin{bmatrix} 5 \\ 5 \end{bmatrix} \quad (142)$$

$$= \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad (143)$$

Similar calculation confirms $\mathbf{u}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$

# SVD in Navigation and Coordinate Transformations - I

**Coordinate Transformation Context**
When $\mathbf{A}$ represents a transformation matrix:

- $\mathbf{V}$ identifies principal input directions
- $\mathbf{\Sigma}$ quantifies distortion along each direction
- $\mathbf{U}$ shows where those directions map to

**Comparison with Eigendecomposition**

**Rigid Transformations**
For rotation matrices $\mathbf{R}$:

- All singular values = 1
- $\mathbf{U}$ and $\mathbf{V}$ encode rotation components
- Can detect if matrix is pure rotation
- SVD can project any matrix to closest rotation matrix

**Navigation Application Examples**

- **Sensor Calibration**
  - Magnetometer soft-iron corrections
  - IMU scaling and cross-axis sensitivity
  - Camera intrinsic parameter estimation

| Eigendecomposition | SVD |
|---|---|
| $\mathbf{A} = \mathbf{Q\Lambda Q}^{-1}$ | $\mathbf{A} = \mathbf{U\Sigma V}^{T}$ |
| Requires square matrix | Works for any matrix |
| May not exist | Always exists |
| Same input/output space | Can connect different spaces |

# SVD in Navigation and Coordinate

1. **GNSS Position Estimation**
   - DOP (Dilution of Precision) analysis:
     - Singular values of geometry matrix reveal positioning accuracy
     - Smaller values indicate higher sensitivity to errors
   - Weighted least-squares solutions:
     - SVD provides stable solution even with near-singular matrices
     - SVD-based pseudoinverse:

$$\mathbf{A}^{+} = \mathbf{V}\boldsymbol{\Sigma}^{+}\mathbf{U}^{T}$$

   - Error ellipsoid orientation and size:
     - Singular vectors indicate principal error directions
     - Singular values proportional to error magnitudes

2. **Map/Terrain Matching**
   - Finding optimal alignment between maps
   - Extracting principal terrain features
   - Reducing dimensionality of terrain data

3. **INS Mechanization**
   - Stable integration of rotation matrices
   - Detecting and correcting numerical errors
   - Analysis of error propagation in inertial solutions
   - Correcting accumulated drift in attitude matrices

**Key Course Connection**

- SVD provides mathematical tools to understand linear transformations in navigation
- Essential for robust implementation of navigation algorithms
- Bridges mathematical theory with practical sensor processing

# SVD for Linear

### Handling Ill-Conditioned Systems

- For small singular values, reciprocals become very large
- Can lead to numerical instability when $\sigma_i \approx 0$
- Solutions:
  - — **Truncated SVD:** Set all $\sigma_i <$ threshold to zero
  - — **Tikhonov regularization:** Replace $\sigma_i^{-1}$ with $\frac{\sigma_i}{\sigma_i^2+\alpha^2}$
- Regularized solution:

$$\mathbf{x}_\alpha = \sum_{i=1}^{r} \frac{\sigma_i}{\sigma_i^2 + \alpha^2} \left(\mathbf{u}_i^T \mathbf{b}\right) \mathbf{v}_i \qquad (149)$$

where $\alpha$ is the regularization parameter

### Navigation Applications

- **GNSS Positioning**
  - — Poor satellite geometry can create ill-conditioned systems
  - — Regularization improves solution stability
  - — Truncated SVD can remove weak satellite contributions
  - — Reveals which measurements contribute most to solution
- **Sensor Calibration**
  - — Determining sensor biases and scale factors
  - — Handling coplanar or near-coplanar sensor arrangements
  - — Resolving ambiguities in calibration data
- **Estimation Problems**
  - — State estimation with redundant or deficient measurements
  - — Handling near-singular estimation problems

# 2 × 2 Example of

**Simple Example**

Consider the matrix:

$$\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 3 & -5 \end{pmatrix} \tag{150}$$

**Steps:**

1. Compute $\mathbf{A}^T\mathbf{A}$ :

$$\mathbf{A}^T\mathbf{A} = \begin{pmatrix} 4 & 3 \\ 0 & -5 \end{pmatrix}\begin{pmatrix} 4 & 0 \\ 3 & -5 \end{pmatrix} \tag{151}$$

$$= \begin{pmatrix} 25 & -15 \\ -15 & 25 \end{pmatrix} \tag{152}$$

2. Find eigenvalues by solving $det(\mathbf{A}^T\mathbf{A} - \lambda\mathbf{I}) = 0$

$$\lambda_1 = 40, \lambda_2 = 10 \tag{153}$$

3. Singular values are square roots of eigenvalues:

$$\sigma_1 = \sqrt{40} \approx 6.32, \sigma_2 = \sqrt{10} \approx 3.16$$

4. Find eigenvectors of $\mathbf{A}^T\mathbf{A}$ (columns of $\mathbf{V}$):

$$\mathbf{V} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \tag{155}$$

5. Compute **U** columns using:

$$\mathbf{u}_i = \frac{1}{\sigma_i}\mathbf{A}\mathbf{v}_i \tag{156}$$

Computing $\mathbf{u_1}$ and $\mathbf{u_2}$ explicitly:

$$\mathbf{u_1} = \frac{1}{6.32}\mathbf{A}\mathbf{v}_1 = \begin{pmatrix} 0.449 \\ 0.894 \end{pmatrix} \tag{157}$$

$$\mathbf{u_2} = \frac{1}{3.16}\mathbf{A}\mathbf{v}_2 = \begin{pmatrix} -0.894 \\ 0.449 \end{pmatrix} \tag{158}$$

6. Final SVD:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \tag{159}$$

where $\mathbf{\Sigma} = \text{diag}(6.32, 3.16)$

# Quick Quiz: SVD Properties



**Take a moment to check your understanding:**

1. Which of these is TRUE about the singular values of a matrix?
   A) They are always positive or zero
   B) They are the square roots of eigenvalues of $\mathbf{A}$
   C) Their number equals the smaller dimension of the matrix
   D) They are always the same as the eigenvalues

2. The condition number of a matrix can tell us:
   A) The exact error in numerical solutions
   B) How sensitive solutions are to changes in input data
   C) The rank of the matrix precisely
   D) Whether the matrix is symmetric or not

3. What makes SVD particularly valuable for navigation problems?
   A) It always produces a unique solution
   B) It works for any matrix shape and reveals rank deficiencies
   C) It is the fastest decomposition method
   D) It requires the least memory

# Quick Quiz: SVD Properties -

1. Which of these is TRUE about the singular values of a matrix?
   **Answer: A) They are always positive or zero**
   - Singular values are, by definition, non-negative
   - They are square roots of eigenvalues of $\mathbf{A}^T\mathbf{A}$ ,not of $\mathbf{A}$
   - The number of non-zero singular values equals the rank of the matrix
   - For non-symmetric matrices, singular values differ from eigenvalue magnitudes

2. The condition number of a matrix can tell us:
   **Answer: B) How sensitive solutions are to changes in input data**
   - Condition number $\kappa$ measures how errors in input propagate to output
   - Relative error in output can be up to $\kappa$ times relative error in input
   - High condition number indicates an ill-conditioned problem
   - Small changes in $\mathbf{b}$ can cause large changes in the solution $\mathbf{x}$

# Quick Quiz: SVD Properties - Solutions

3. What makes SVD particularly valuable for navigation problems?
   **Answer: B) It works for any matrix shape and reveals rank deficiencies**
   - SVD applies to any rectangular matrix, unlike eigendecomposition
   - It immediately reveals rank deficiency through zero singular values
   - In GNSS positioning, rank deficiency indicates insufficient satellite geometry
   - It provides a solution that minimizes error even in underdetermined systems
   - SVD handles both overdetermined systems (more measurements than unknowns) and underdetermined systems (fewer measurements than unknowns)

SVD is the most general and robust matrix decomposition, making it ideal for complex navigation problems with varying geometry and measurement availability.

# Differentiation Review: Basic

**Fundamental Rules**

**1. Power Rule:**

$$f(x) = x^n \Rightarrow f'(x) = nx^{n-1} \tag{160}$$

**2. Chain Rule:**

$$f\big(g(x)\big) \Rightarrow f'\big(g(x)\big) \cdot g'(x) \tag{161}$$

**3. Product Rule:**

$$(uv)' = u'v + uv' \tag{162}$$

**4. Quotient Rule:**

$$\left(\frac{u}{v}\right)' = \frac{u'v - uv'}{v^2} \tag{163}$$

**Common Functions**

**Trigonometric:**

$$\frac{d}{dx}(\sin x) = \cos x \tag{164}$$

$$\frac{d}{dx}(\cos x) = -\sin x \tag{165}$$

**Exponential & Logarithmic:**

$$\frac{d}{dx}(e^x) = e^x \tag{166}$$

$$\frac{d}{dx}(\ln x) = \frac{1}{x} \tag{167}$$

$$\frac{d}{dx}\left(\sqrt{x}\right) = \frac{1}{2\sqrt{x}} \tag{168}$$

$$\frac{d}{dx}(x^{-1}) = -x^{-2} \tag{169}$$

# Differentiation Review: Navigation

**Example 1: Range Function**
Distance to a fixed point:

$$r = \sqrt{(x - x_0)^2 + (y - y_0)^2} \tag{171}$$

Time derivative (using chain rule):

$$\frac{dr}{dt} = \frac{(x - x_0)\dot{x} + (y - y_0)\dot{y}}{r} \tag{172}$$

**Physical meaning:**

- Radial velocity component
- Positive: moving away
- Negative: approaching

**Example 2: Position to Velocity**
Given position:

$$x(t) = v_0 t + \frac{1}{2} a t^2 \tag{173}$$

Velocity:

$$v(t) = \frac{dx}{dt} = v_0 + at \tag{174}$$

**Example 3: 3D Satellite Range**
Distance to satellite:

$$\rho = \sqrt{(x - x_s)^2 + (y - y_s)^2 + (z - z_s)^2} \tag{175}$$

Partial derivatives:

$$\frac{\partial \rho}{\partial x} = \frac{x - x_s}{\rho} \tag{176}$$

$$\frac{\partial \rho}{\partial y} = \frac{y - y_s}{\rho} \tag{177}$$

$$\frac{\partial \rho}{\partial z} = \frac{z - z_s}{\rho} \tag{178}$$

Gradient vector:

$$\nabla \rho = \frac{1}{\rho} \begin{bmatrix} x - x_s \\ y - y_s \\ z - z_s \end{bmatrix} \tag{179}$$

# Practice Problems: Test Your

Work through these problems relevant to navigation:

**1. Distance to Satellite**: Given satellite position $(x_s, y_s, z_s) = (20000, 15000, 25000)$ km and receiver at $(x, y, z)$:

$$d = \sqrt{(x - 20000)^2 + (y - 15000)^2 + (z - 25000)^2} \quad (180)$$

Find $\frac{\partial d}{\partial x}$, $\frac{\partial d}{\partial y}$ and $\frac{\partial d}{\partial z}$.

**2. Bearing Angle:** The bearing from position $(x1, y1)$ to $(x2, y2)$ is:

$$\beta = \arctan\left(\frac{x_2 - x_1}{y_2 - y_1}\right) \quad (181)$$

Find $\frac{\partial \beta}{\partial x_1}$ and $\frac{\partial \beta}{\partial y_1}$

**3. Doppler Shift:** The Doppler frequency shift for a moving receiver is:

$$f_D = \frac{f_0}{c} \cdot \frac{d}{dt}\left(\sqrt{\left(x(t) - x_s\right)^2 + \left(y(t) - y_s\right)^2 + \left(z(t) - z_s\right)^2}\right) \quad (182)$$

Express this in terms of velocity components $v_x$ ,$v_y$ , $v_z$ .

# Solutions to Practice

**1. Distance Derivatives:**

$$\frac{\partial d}{\partial x} = \frac{x - 20000}{d} \quad (183)$$

$$\frac{\partial d}{\partial y} = \frac{y - 15000}{d} \quad (184)$$

$$\frac{\partial d}{\partial z} = \frac{z - 25000}{d} \quad (185)$$

These form a unit vector pointing from satellite to receiver!

**2. Bearing Derivatives:**

$$\frac{\partial \beta}{\partial x_1} = \frac{-(y_2 - y_1)}{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (186)$$

$$\frac{\partial \beta}{\partial y_1} = \frac{x_2 - x_1}{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (187)$$

**Note**: These show how bearing changes as we move our position.

**3. Doppler Shift:**

Using chain rule:

$$f_D = \frac{f_0}{c} \cdot \frac{1}{d}\left[(x - x_s)v_x + (y - y_s)v_y + (z - z_s)v_z\right] \quad (188)$$

This can be written as:

$$f_D = \frac{f_0}{c} \cdot \mathbf{e} \cdot \mathbf{v} \quad (189)$$

where **e** is the unit vector from receiver to satellite and **v** is the velocity vector.

**Physical Insight:**

- Doppler is proportional to radial velocity
- Maximum when moving directly toward satellite
- Zero when moving perpendicular to line-of-sight

# From Derivatives to Jacobian Matrices - I: Single

**Single Variable Differentiation**

For a function $f: \mathbb{R} \to \mathbb{R}$:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \tag{191}$$

**Example:** Distance from beacon

$$f(x) = \sqrt{(x - x_0)^2 + y_0^2} \tag{192}$$

$$f'(x) = \frac{x - x_0}{\sqrt{(x - x_0)^2 + y_0^2}} \tag{193}$$

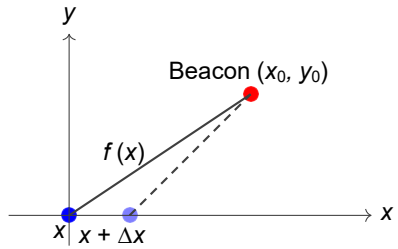
**Linear Approximation:**

$$f(x + \Delta x) \approx f(x) + f'(x) \cdot \Delta x \tag{194}$$

The derivative tells us:

- Rate of change
- Sensitivity to input changes
- Direction of steepest increase

**Navigation Example: 1D Positioning**




**Physical Meaning:**

- $f(x)$: measured distance
- $f'(x)$ :how distance changes as receiver moves
- Sign indicates approaching/receding
- Magnitude shows sensitivity

# From Derivatives to Jacobian Matrices - II: Multivariable

**Vector-Valued Functions**
For $\mathbf{f}: \mathbb{R}^n \to \mathbb{R}^m$ :

$$f(\mathbf{x}) = \begin{bmatrix} f_1(x_1, x_2, \ldots x_n) \\ f_2(x_1, x_2, \ldots x_n) \\ \vdots \\ f_m(x_1, x_2, \ldots x_n) \end{bmatrix} \tag{195}$$

**The Jacobian Matrix:**

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \tag{196}$$

Each row: gradient of one output function
Each column: sensitivity to one input variable

**Example: 2D to 2D Transformation**
Polar to Cartesian conversion:

$$f(r, \theta) = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} r\cos\theta \\ r\sin\theta \end{bmatrix} \tag{197}$$

Jacobian matrix:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial \theta} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & -r\sin\theta \\ \sin\theta & r\cos\theta \end{bmatrix} \tag{198}$$

**Linear Approximation:**

$$f(x + \Delta x) \approx f(x) + J\Delta x \tag{199}$$

**Properties:**

- $det(J) = r$ (area scaling factor)
- Singular when $r = 0$ (at origin)
- Shows how errors propagate

# Jacobian Example 1: Simple 2D

**Function Definition:**

$$f(x,y) = \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} x^2 + 2y \\ 3x - y^2 \end{bmatrix} \quad (200)$$

**Step 1: Find partial derivatives**

$$\frac{\partial u}{\partial x} = 2x, \frac{\partial u}{\partial y} = 2 \quad (201)$$

$$\frac{\partial v}{\partial x} = 3, \frac{\partial v}{\partial y} = -2y \quad (202)$$

**Step 2: Form Jacobian matrix**

$$\mathbf{J} = \begin{bmatrix} 2x & 2 \\ 3 & -2y \end{bmatrix} \quad (203)$$

**Numerical Example:**
At point $(x, y) = (1, 2)$:

$$f(1,2) = \begin{bmatrix} 1^2 + 2(2) \\ 3(1) - 2^2 \end{bmatrix} = \begin{bmatrix} 5 \\ -1 \end{bmatrix} \quad (204)$$

Jacobian at (1, 2):

$$\mathbf{J}(1,2) = \begin{bmatrix} 2(1) & 2 \\ 3 & -2(2) \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 3 & -4 \end{bmatrix} \quad (205)$$

**Linear approximation:** Near (1, 2), for small $\Delta x, \Delta y$:

$$f(1 + \Delta x, 2 + \Delta y) \approx \begin{bmatrix} 5 \\ -1 \end{bmatrix} + \begin{bmatrix} 2 & 2 \\ 3 & -4 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (206)$$

# Jacobian Example 2: Product and Ratio

**Function Definition:**

$$f(x,y) = \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} xy \\ x/y \end{bmatrix} \quad (207)$$

**Partial derivatives:**

$$\frac{\partial u}{\partial x} = y, \frac{\partial u}{\partial y} = x \quad (208)$$

$$\frac{\partial v}{\partial x} = \frac{1}{y}, \frac{\partial v}{\partial y} = -\frac{x}{y^2} \quad (209)$$

**Jacobian matrix:**

$$\mathbf{J} = \begin{bmatrix} y & x \\ 1/y & -x/y^2 \end{bmatrix} \quad (210)$$

**Numerical Example:**

At point $(x, y) = (6, 3)$:

$$f(6,3) = \begin{bmatrix} 6 \times 3 \\ 6/3 \end{bmatrix} = \begin{bmatrix} 18 \\ 2 \end{bmatrix} \quad (211)$$

Jacobian at (6, 3):

$$\mathbf{J}(6,3) = \begin{bmatrix} 3 & 6 \\ 1/3 & -6/9 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 0.333 & -0.667 \end{bmatrix} \quad (212)$$

**Check with small perturbation:**

$$f(6.1,3.1) = \begin{bmatrix} 18.91 \\ 1.968 \end{bmatrix}$$

Linear approximation gives:

$$\begin{bmatrix} 18 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 & 6 \\ 0.333 & -0.667 \end{bmatrix} \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 18.9 \\ 1.967 \end{bmatrix}$$

# Jacobian Example 3: Three Variables to Two

**Function Definition:**

$$f(x,y,z) = \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} x+y+z \\ x^2+y^2+z^2 \end{bmatrix} \quad (213)$$

**Partial derivatives:**

$$\frac{\partial u}{\partial x} = 1, \frac{\partial u}{\partial y} = 1, \frac{\partial u}{\partial z} = 1 \quad (214)$$

$$\frac{\partial v}{\partial x} = 2x, \frac{\partial v}{\partial y} = 2y, \frac{\partial v}{\partial z} = 2z \quad (215)$$

**Jacobian matrix (2×3):**

$$\mathbf{J} = \begin{bmatrix} 1 & 1 & 1 \\ 2x & 2y & 2z \end{bmatrix} \quad (216)$$

Note: Non-square Jacobian!

**Numerical Example:**
At point (*x*, *y*, *z*) = (1, 2, 3):

$$f(1,2,3) = \begin{bmatrix} 1+2+3 \\ 1^2+2^2+3^2 \end{bmatrix} = \begin{bmatrix} 6 \\ 14 \end{bmatrix} \quad (217)$$

Jacobian at (1, 2, 3):

$$\mathbf{J}(1,2,3) = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 6 \end{bmatrix} \quad (218)$$

**Interpretation:**

- First row: equal sensitivity to all inputs
- Second row: *z* has strongest effect on *v*
- Can't invert (not square) For $\Delta x = [0.1, 0.1, 0.1]^T$:

$$\Delta f = \mathbf{J}\Delta x = \begin{bmatrix} 0.3 \\ 1.2 \end{bmatrix} \quad (219)$$

# Jacobian Calculation: Summary and

**Steps to Calculate Jacobian:**

1. Identify input and output dimensions
2. Calculate all partial derivatives
3. Arrange in matrix form:

$$\mathbf{J}_{ij} = \frac{\partial f_i}{\partial x_j} \tag{220}$$

4. Evaluate at specific point if needed

**Quick Practice:** Find the Jacobian of:

$$f(x, y) = \begin{bmatrix} 2x + y \\ x - 3y \\ xy \end{bmatrix} \tag{221}$$

**Solution:**

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \\ \frac{\partial f_3}{\partial x} & \frac{\partial f_3}{\partial y} \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & -3 \\ y & x \end{bmatrix} \tag{222}$$

At point $(x, y) = (4, 2)$:

$$\mathbf{J}(4,2) = \begin{bmatrix} 2 & 1 \\ 1 & -3 \\ 2 & 4 \end{bmatrix} \tag{223}$$

**Key Points:**

- Size: 3×2 (3 outputs, 2 inputs)
- Some entries constant, some variable
- Each row = gradient of one output

# From Derivatives to Jacobian Matrices - III: Navigation

**GNSS Positioning Example**

Range measurements from satellites:

$$\rho_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} \quad (224)$$

For $m$ satellites, Jacobian is $m \times 3$:

$$\mathbf{H} = \begin{bmatrix} \frac{x - x_1}{\rho_1} & \frac{y - y_1}{\rho_1} & \frac{z - z_1}{\rho_1} \\ \frac{x - x_2}{\rho_2} & \frac{y - y_2}{\rho_2} & \frac{z - z_2}{\rho_2} \\ \vdots & \vdots & \vdots \\ \frac{x - x_m}{\rho_m} & \frac{y - y_m}{\rho_m} & \frac{z - z_m}{\rho_m} \end{bmatrix} \quad (225)$$

Each row: unit vector from receiver to satellite

**Linearized Navigation Equation:**

$$\Delta\rho = \mathbf{H}\Delta x + \varepsilon \quad (226)$$

**Why Jacobians Matter in Navigation**

1. **Sensitivity Analysis:**
   - How much does position error affect range?
   - Which direction gives best accuracy?
2. **Geometric Interpretation:**
   - Each row is a direction vector
   - Good geometry = diverse directions
   - Poor geometry = similar directions
3. **Simple Example - 2D Case:**

For 2 beacons at $(0, 0)$ and $(10, 0)$:

$$\mathbf{H} = \begin{bmatrix} \frac{x}{\rho_1} & \frac{y}{\rho_1} \\ \frac{x - 10}{\rho_2} & \frac{y}{\rho_2} \end{bmatrix} \quad (227)$$

**Key Insight:**

- Jacobian shows measurement sensitivity
- Links single derivatives to systems

# Summary and Key



**Advanced Matrix Properties**

- Eigenvalues reveal system characteristics
- Positive definiteness ensures stability
- Condition number predicts numerical behavior
- Matrix decompositions offer different advantages

**Matrix Decompositions**

- Different decompositions for different problems
- Selection based on problem structure
- Navigation applications have specific requirements
- Computational efficiency vs. numerical stability

**Singular Value Decomposition**

- Universal decomposition for any matrix
- Reveals key matrix properties (rank, condition)
- Provides stable solutions to linear systems
- Handles rank deficiency naturally
- Offers regularization methods for ill-conditioned problems

**Navigation Applications**

- GNSS position estimation
- Sensor calibration
- State estimation
- Data compression
- System analysis
- Filtering algorithms

# Matrix Calculation Tools and

**Software Libraries**
**General-purpose mathematical:**

- **MATLAB/Octave**
  - Built-in matrix operations
  - eig, svd, inv, pinv
  - Optimized for matrix manipulations
- **Python Ecosystem**
  - NumPy: np.linalg module
  - SciPy: Advanced decompositions
  - SymPy: Symbolic matrix calculations
- **Specialized Libraries**
  - LAPACK/BLAS: Industry standard
  - Eigen (C++): Template library
  - Armadillo, Blaze, Eigen3

**Navigation-Specific Tools**
**Navigation libraries:**

- **RTKLIB**: GNSS positioning algorithms
- **GeographicLib**: Geodetic transformations
- **GPSTK**: GPS toolkit with matrix utilities
- **ROS tf2**: Transformation framework

**Implementation Considerations:**

- **Embedded systems**:
  - Fixed-point vs. floating-point
  - Memory constraints
  - Real-time requirements
- **Numerical stability**:
  - Avoid direct inversion when possible
  - Use decompositions (LU, QR, SVD)
  - Consider condition number
  - Regularization techniques

# Matrix Computation Tutorial - Part 1:



**Tutorial Overview**

**Objective:** Compare computational accuracy and efficiency of matrix operations between C++/Eigen and Python/NumPy

**Learning Outcomes:**

- Understand performance differences between C++ and Python for matrix operations
- Identify accuracy trade-offs in different numerical implementations
- Learn to properly benchmark numerical algorithms
- Apply findings to navigation algorithm implementation decisions

**Setup Instructions:**

1. Download starter code from course website
2. Install required libraries:
   - C++: Eigen, Chrono
   - Python: NumPy, SciPy, time, memoryprofiler
3. Follow README instructions to compile C++ code
4. Use provided scripts to run comparison tests

# Matrix Computation Tutorial - Part 2: Operations to



**Operations to Implement:**

- Matrix multiplication
  - Standard matrix product
  - Element-wise multiplication
- Matrix inversion
  - Direct inversion
  - Using decompositions
- Eigenvalue decomposition
  - Full decomposition
  - Only eigenvalues
- Singular Value Decomposition
  - Full vs. reduced SVD
- Least squares solution

**Testing Parameters:**

**Matrix dimensions:**

- Small: $10 \times 10$
- Medium: $100 \times 100$
- Large: $1000 \times 1000$

**Data types:**

- Single precision (float)
- Double precision (double)

**Matrix types:**

- Random matrices
- Nearly singular matrices
- Structured matrices (e.g., diagonal dominant)
- Orthogonal matrices

**Number of iterations:**

- 10, 100, 1000 (operation dependent)

# Matrix Computation Tutorial - Part 3:



**Metrics to Measure:**

- **Execution time**
  - Wall clock time
  - CPU time
- **Memory usage**
  - Peak memory
  - Memory growth
- **Numerical accuracy**
  - Frobenius norm error
  - Maximum element-wise error
  - Relative error
- **Scaling behavior**
  - Time vs. matrix size
  - Memory vs. matrix size

**Accuracy Comparison:**

- Use higher precision library for reference solutions
- Test with well-conditioned and ill conditioned matrices
- Verify correctness of operations ($\mathbf{A} \times \mathbf{A}^{-1} = \mathbf{I}$)

**Performance Analysis:**

- Use consistent hardware for all tests
- Disable background processes during benchmarking
- Run multiple iterations and report statistics
- For C++: test with different compiler optimizations
- For Python: compare NumPy with native implementation

# Matrix Computation Tutorial - Part 4: Code



## Sample Code Snippets

### C++/Eigen:

```cpp
#include <Eigen/Dense>
#include <chrono>
#include <iostream>

int main() {
 // Create random matrix
 Eigen::MatrixXd A = Eigen::MatrixXd::Random(100, 100);

 // Time SVD decomposition
 auto start = std::chrono::high_resolution_clock::now();
 Eigen::JacobiSVD<Eigen::MatrixXd> svd(A);
 auto end = std::chrono::high_resolution_clock::now();
 std::chrono::duration<double> elapsed = end-start;
 std::cout << "SVD Time: " << elapsed.count()
           << "s" << std::endl;

 return 0;
}
```

### Python/NumPy:

```python
import numpy as np
import time
import memory_profiler

# Create random matrix
A = np.random.rand(100, 100)

# Time SVD decomposition
start = time.time()
u, s, vh = np.linalg.svd(A)
end = time.time()

print(f"SVD Time: {end-start:.6f}s")

# Memory usage can be tracked with:
# @memory_profiler.profile
# def run_svd(A):
#     return np.linalg.svd(A)
```

## Questions?

Dr. Weisong Wen
Assistant Professor

Trustworthy AI and Autonomous Systems Laboratory (TAS Lab)
Department of Aeronautical and Aviation Engineering
The Hong Kong Polytechnic University

**Email**: *welson.wen@polyu.edu.hk*
**Office**: R820 **Phone**: 3400 8234